Higher National Diploma in Network Engineering, Year 2
BSc (Hons) Ethical Hacking & Network Security, Year 2

PRINCIPLES OF INFORMATION SECURITY

Fingerprints: hashing and integrity



Day 3 - Morning

Running case: Taxila College, a fictional college

Niranga Dharmaratna
School of Computing & Engineering
National Institute of Business Management

Today

Spot a one-mark change in a file, and keep passwords safe even if the store is stolen.

- A fingerprint for any data - 35 min
- One change, a whole new fingerprint - 25 min
- One way only - 25 min
- Storing passwords safely - 35 min
- A seal both sides can check - 30 min



Today serves two outcomes

LO1 Explain the fundamentals of information security.

LO2 Discuss the key characteristics of cryptography and cryptographic systems.

LO3 Analyze the principles of encryption, decryption, hashing, and data integrity.

LO4 Outline the process of ensuring digital security.

LO5 Describe the fundamentals of threats and countermeasures.

LO6 Explain the fundamentals and process of business continuity planning.

Two problems from last semester

RECALL

Integrity means data has not been changed without permission.

- A student's final grade was found **higher** than the lecturer entered
- Nothing showed when it changed
- Staff and students **reuse** their passwords
- If the SIS database were stolen tonight, what would the thief read?

Yesterday's recipe, SHA-256, answers both. Today: what it is.

IT

HEAD OF IT

If someone copied the SIS database tonight, I want them to find nothing they can sign in with.

The Head of IT, on the third day of your placement.

CONCEPT 1

A fingerprint for any data

A published results file carries its digest so anyone can check it is unchanged.

TRY IT FIRST

Any input, a digest of the same fixed length.

35 min

A fingerprint for any data

- A **hash function** takes an input of any size
- It returns a **digest** of one fixed length
- The same input always gives the same digest
- SHA-256: 256 bits, written as **64** hexadecimal characters

SHA-256 of `a`: `ca978112...afee48bb`, 64 characters.

Demonstration: any size in, one size out

1. Fingerprint one letter, then a results line
2. Fingerprint a paragraph, then a whole megabyte
3. Read the size and the digest length each time
4. Fingerprint the same input again and compare

A million times bigger, and still 64 characters.

A fingerprint identifies; it does not hide

- A hash is **not** encryption: the data stays as readable as before

YOU ALREADY KNOW THIS

A fingerprint identifies one person. Nobody can grow the person back from it.

MAPPING

the fingerprint = the digest · **you cannot grow a person back from a fingerprint =** you cannot reverse a hash

× COMMON MISTAKE

A longer file gives a longer digest.

The digest is a fixed length whatever the input size.

CONCEPT 2

One change, a whole new fingerprint

A grade file altered by one mark has a completely different digest.

TRY IT FIRST

One character changed, the whole digest changes.

25 min

Demonstration: one change, a new fingerprint

1. Compare the digests of two lines one mark apart
2. Count the characters that differ
3. Read the grid of 256 bits
4. Change one character at random, again and again

About half the bits flip, every time.

The avalanche effect

- `COHNDNE999X000,50` gives `84a521d5...4b4e3f`
- `COHNDNE999X000,51` gives `b2d1fd0d...e0ef8a`
- **62** of 64 characters differ; 127 of 256 bits flipped
- No visible link between the two digests

YOU ALREADY KNOW THIS

One wrong note in a song you know makes the whole tune sound wrong at once.

MAPPING

one changed note = one changed character · **the whole tune sounds different =** the whole digest changes

A digest is a tamper check

- Record the digest when you publish the file
- Later, take the digest again and compare
- Match: the file is the same, byte for byte
- Any change, even one character, and they differ

The recorded digest must be kept where the changer cannot reach it.

T TAXILA

A grade file altered by one mark has a completely different digest.

CONCEPT 3

One way only

Once Taxila hashes its passwords, a stolen database reveals digests, not passwords.

TRY IT FIRST

A digest cannot be reversed, unlike encryption.

25 min

Built to run forwards only

RECALL

A key makes encryption reversible; hashing works the other way.

- **Preimage resistance**: given a digest, you cannot find an input that made it
- **Collision resistance**: you cannot find two inputs with the same digest
- Encryption reverses with a key; a hash has **no key** and no reverse

Demonstration: one way only

1. Encrypt a line and decrypt it with the key
2. Fingerprint the line and look for a way back
3. Watch twenty guesses: one digest found, one not
4. Hash twenty thousand inputs; find the closest pair

The only way back is to guess, hash and compare.

Hash functions grow old

Older **32 chars**	Older **40 chars**	Current **64 chars**
MD5 Message Digest 5, 128 bits. *Deprecated: collisions are easy to find.*	**SHA-1** Secure Hash Algorithm 1, 160 bits. *Deprecated: collisions have been found.*	**SHA-256** Secure Hash Algorithm, 256 bits. *The current choice.*

Tell them apart by the **length** of the digest alone.

A shredder, not a safe

YOU ALREADY KNOW THIS

A paper shredder turns a page into strips. They came from that page, but nobody can put it back together.

MAPPING

the shredder = the hash · **you cannot reassemble the page =** preimage resistance

T TAXILA

Once Taxila hashes its passwords, a stolen database would reveal digests, not passwords.

× COMMON MISTAKE

A hash is a kind of encryption you can undo.

A hash cannot be reversed; there is no key that opens it.

CONCEPT 4

Storing passwords safely

Two staff who chose the same password must not share a stored value.

TRY IT FIRST

The same password with two salts stores differently.

35 min

Never store the password itself

RECALL

Only encryption needs a key, and whoever holds the key can reverse it.

- A system stores a **digest** of each password
- At sign-in it hashes what you typed and compares
- Unsalted digests can still be matched in advance

A table built in advance

- A **dictionary** is a list of common passwords
- Hash every word in advance and keep the results
- A **rainbow table** is a very large table of this kind
- A stolen digest is looked up and **matched**

The table never reverses a hash: it only matches one.

Demonstration: a pinch of salt

1. Hash a common password and look it up
2. Give two staff the same password and two salts
3. Look up both stored values
4. Time one SHA-256 against one slow hash

Same password, two salts: two values, and neither in the table.

A salt for every user

- A **salt** is added to the password before hashing
- Each user has a different salt, stored **beside** the digest
- Nobody can build a table for every salt in advance

YOU ALREADY KNOW THIS

Two cooks follow the same recipe, and each adds a different secret spice.

MAPPING

the recipe = the password · **the unique spice =** the salt · **the dishes now taste different =** the stored digests differ

Salt and pepper

Salt

- Different for every user
- Stored beside the digest
- Need not be secret: it must be unique
- Defeats tables built in advance

Pepper

- The same for every user
- Kept apart from the database
- A further secret
- A stolen database alone is not enough

Encryption is the wrong tool: whoever holds the key reads every password.

Slow on purpose

- Fast hashes let a guesser try **millions** of passwords a second
- **bcrypt**, **Argon2** and **PBKDF2** repeat their work many times
- PBKDF2: the Password-Based Key Derivation Function 2
- A user waits once; a guesser waits on every guess

Speed is good for files and bad for passwords.

CONCEPT 5

A seal both sides can check

The examinations office knows a key-holder produced the results file unchanged.

TRY IT FIRST

A keyed seal nobody can forge or check without the key.

30 min

A hash with a secret key mixed in

RECALL

Non-repudiation means a person cannot later deny an action.

- Anyone can remake a plain digest, even the changer
- A hash-based message authentication code (**HMAC**) mixes in a secret key
- Only a key-holder can make or check the **seal**
- The message itself stays readable

Demonstration: a keyed seal

1. Make the office key and seal the results file
2. Check the seal, then change one mark
3. Remake a plain digest and a seal without the key
4. Let Colombo and Kandy each seal the same file

Without the key, nobody can make a seal that matches.

What a plain digest and a seal prove

A plain digest

- Anyone can make it
- Shows no change only if kept safe
- Says nothing about who made it

An HMAC seal

- Only key-holders can make it
- Shows the message is unchanged
- Shows **a** key-holder made it, not which

Day 4: a signature shows which person.

A seal both sides can check

YOU ALREADY KNOW THIS

A tamper strip needs a stamp. The sender and the receiver each hold a copy of the stamp.

MAPPING

the shared stamp = the secret key both sides hold · **a matching strip =** the keyed hash verifies · **either side could have stamped it =** no non-repudiation

T TAXILA

A results file carries a keyed-hash seal, so the office knows a key-holder produced it unchanged.

× COMMON MISTAKE

A keyed-hash seal proves which person sent the message.

Both sides share the key, so it proves only that a key-holder sent it.

Common mistakes

× COMMON MISTAKE

A longer file gives a longer digest.

The digest is a fixed length whatever the input size.

× COMMON MISTAKE

A hash is a kind of encryption you can undo.

A hash cannot be reversed; there is no key that opens it.

× COMMON MISTAKE

Hashing a password is enough without a salt.

Without a salt, a lookup table reverses common passwords at once.

Before the next day

READ	**Security in Computing, section 12.4** Message digests; then section 2.1 on storing passwords.
PRACTICE	**An online bookshop** Practice question 3: password storage, salting, why speed is a weakness.
PACK	**Evidence pack 3** After this afternoon's lab. *Due before the next teaching day.*

Questions & Discussion

This afternoon: fingerprint your own results file and store a password safely.



Disclaimer & Copyright

The information in this presentation is for general teaching purposes only. It is not comprehensive and is not professional advice; any reliance you place on it is at your own risk. I aim to keep it accurate, but I make no representations or warranties of any kind about its completeness, accuracy, reliability or suitability.

Security tools, standards and laws change. Versions, editions and commencement dates in this presentation were checked at the time of writing; check the current documentation before relying on them.

This presentation may refer to third-party websites or content. I do not endorse their views or information, and I am not responsible for their content.

Taxila College is fictional. Its campuses, people, systems and incidents were invented for teaching. Product and tool names, for example Kali Linux, VirtualBox, CyberChef and Wireshark, belong to their owners; this material is not affiliated with or endorsed by any of them.

Practise only on your own virtual machine, the module's provided files and application, and the portal's demos. Testing a system you do not own, without written permission, is an offence under Sri Lanka's Computer Crime Act, No. 24 of 2007.

© Niranga Dharmaratna, School of Computing & Engineering, National Institute of Business Management. Teaching content under CC BY-NC-SA 4.0; code and demonstrations under MIT; Open Sans under the SIL Open Font Licence.

Revision 2f4edeb